Banco de Dados

Linguagem SQL
DDL – Data Definition Language

Agenda

- Introdução
- Diagrama do banco de dados
- Script SQL
  - Create database
  - Create table
  - Alter table
  - Drop database
  - Drop table
- Dúvidas e comentários

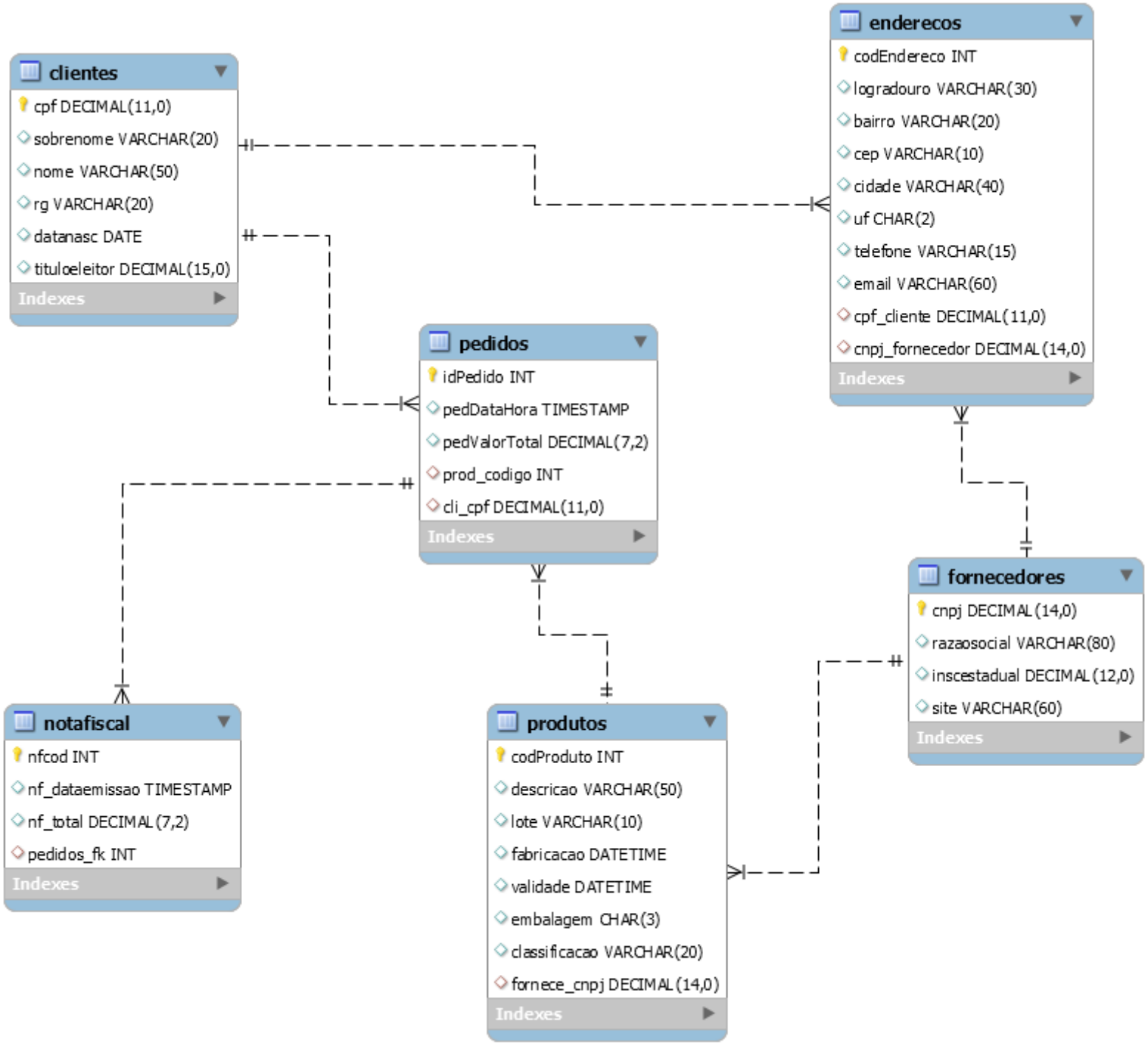
Introdução

Segundo Neto (2003), Structured Query Language (SQL), ou Linguagem de Consulta Estruturada, é a linguagem estruturada padrão para acessar dados num sistema gerenciador de banco de dados. Ela foi criada originalmente no início dos anos 80, pela IBM. Em meados de 1980, a **ANSI** iniciou seus trabalhos de desenvolvimento para padronização de uma linguagem para banco de dados relacionais.

A ANSI e a ISO juntaram esforços e trabalharam num conjunto maior de padronização para o SQL, criando o chamado SQL ANSI-92, ou ainda SQL-92 ou simplesmente SQL2. Um grupo de trabalho estendeu o SQL2 para um novo padrão que incorpora características orientadas a objetos (como, por exemplo, a herança), o SQL3.

Etec
Dr. Adail Nunes da Silva
Taquaritinga

DER - Comércio



Create database

A instrução create database, como o próprio nome sugere, serve para criarmos a base de dados na qual as tabelas serão criadas.

Sua sintaxe é bastante simples. Vejamos, por exemplo, a criação de uma base de dados chamada **comercio**.

```
1 -- criação da base de dados comercio
2 CREATE DATABASE comercio;
3
4 -- Usar a base de dados comercio (base padrão)
5 USE comercio;
```

Create table

Após criarmos a nossa base de dados, criaremos as nossas tabelas. Para isso, faremos uso do comando **create table**.

O comando **create table** permite criarmos e definirmos a estrutura de uma tabela, definindo suas colunas (campos), suas respectivas restrições, além de suas chaves primárias e estrangeiras.

```
6  -- Criação da tabela clientes
7  -- Create table clientes
8  Create table if not exists clientes (
9      cpf decimal(11) not null,
10     sobrenome varchar(20),
11     nome varchar(50),
12     rg varchar(20),
13     datanasc date,
14     tituloeleitor decimal(15),
15     primary key(cpf)
16  );
```

Create table

Criando a tabela **fornecedores** e aplicando a **constraint** (restrição) do campo **cnpj** como chave primária.

```
17
18 -- Create table fornecedores
19 -- CNPJ 111.111.111/1111
20 Create table if not exists fornecedores
21 (     cnpj decimal(14) not null,
22      razaosocial varchar(80),
23      inscestadual decimal(12),
24      site varchar(60),
25     primary key(cnpj)
26  );
27
```

Create table

Criando a tabela **enderecos** com os campos definidos, além de criar a **constraint** (restrição) do campo **codEndereco** como chave primária os campos **cpf_cliente** e **cnpj_fornecedor** como chaves estrangeiras.

```
28 -- Create table enderecos
29 Create table if not exists enderecos
30 (    codEndereco int(3) not null AUTO_INCREMENT,
31      logradouro varchar(30),
32      bairro varchar(20),
33      cep varchar(10),
34      cidade varchar(40),
35      uf char(2),
36      telefone varchar(15),
37      email varchar(60),
38      cpf_cliente decimal(11),
39      cnpj_fornecedor decimal(14),
40      primary key(codEndereco),
41      foreign key(cpf_cliente) REFERENCES clientes(cpf),
42      foreign key(cnpj_fornecedor) REFERENCES fornecedores(cnpj)
43 );
```

Alter table

Segundo Pazin (2003), o comando **alter table** permite alterar a estrutura de uma tabela acrescentando, alterando, retirando e alterando nomes, formatos das colunas e a integridade referencial definidas em uma determinada tabela.

Sintaxe do comando **Alter table**:

ALTER TABLE nome_da_tabela
    DROP nome_coluna
    ADD nome_da_coluna tipo_de_dado (NOT NULL)

    MODIFY nome_da_coluna tipo_de_dado (NOT NULL);

Exemplos do comando Alter


```sql
-- Altera a tabela clientes excluindo o campo datanasc
ALTER TABLE clientes
      DROP datanasc;

-- Altera a tabela clientes adicionando o campo nascimento
ALTER TABLE clientes
      ADD nascimento date;

-- Altera a tabela clientes modificando o campo nome para varchar(80)
-- e torna o campo obrigatório
ALTER TABLE clientes
      MODIFY nome varchar(80) NOT NULL;
```

Comando drop

O comando **drop table** serve para destruirmos uma tabela. Se, por exemplo, precisássemos destruir a tabela **clientes**, usaríamos o seguinte comando:

```
14 -- dropa (exclui) a tabela clientes
15 DROP TABLE clientes;
```

O comando **drop database** serve para destruirmos a base de dados. Se, por exemplo, precisássemos destruir o banco de dados **comercio**, usaríamos o seguinte comando:

```
17 -- dropa (exclui) a base de dados
18 DROP DATABASE comercio;
```

Linguagem DML

- Comandos DML (Data Manipulation Language
  - Insert
  - Insert posicional
  - Insert declarativo
- Dicas
- Laboratório
- Dúvidas e comentários

Manipulando dados

Conforme apresentado, na linguagem SQL existem instruções para definição de dados (DDL), e instruções para manipulação de dados (DML).

Conhecemos, até agora, alguns comandos DDL e, nos próximos slides, conheceremos instruções de manipulação como o comando **INSERT** e outros.

DML

INSERT
UPDATE
DELETE
SELECT

Inserindo dados na tabela

Para incluirmos dados em uma tabela, utilizamos a instrução **insert into**. Este comando permite inserir um ou vários registros em uma tabela do banco de dados.

MySQL

INSERT

```
INSERT INTO table ( col1, col2 ) VALUES ( 'val1', 'val2' )
```

SELECT

```
SELECT * FROM table WHERE col1 = 'val1' AND col2 = 'val2'
```

UPDATE

```
UPDATE table SET col1 = 'val1', col2 = 'val2' WHERE col3 = 'val1'
```

DELETE

```
DELETE FROM table WHERE col1 = 'val1'
```

Insert posicional

O comando INSERT é classificado como posicional quando não especifica a lista de colunas que receberão os dados de VALUES Nesse caso, a lista de valores precisa conter todos os campos na ordem física em que foram criadas no comando CREATE TABLE.

```
-- Insert posicional
INSERT INTO clientes VALUES
(50956475329, 'Melo', 'Breno Oliver', '351224683', '1995-10-13', 12345678912345),
(33071211627, 'Paz', 'Fábio Luan da', '457732507', '1982-04-24', 12365498745633),
(11882181077, 'Silveira', 'Tereza Manuela Rita', '122931737', '1943-07-20', 12332145698777),
(13335743393, 'Castro', 'Gabrielly Ester Sabrina', '250821722', '1950-11-26', 12344578965254),
(86532048224, 'Santos', 'Elias Julio', '219901053', '1998-10-25', 1239986657411),
(95640167114, 'Fernandes', 'Helena Luana', '129121988', '1976-03-22', 12312312348874),
(80050797450, 'Neves', 'Bruna Clara', '331973091', '1958-05-04', 12399845676355),
(43343748161, 'Silva', 'Alessandra Stefany', '218698252', '1987-07-21', 12377745896313),
(66725468713, 'Gomes', 'Pedro Henrique', '146895654', '1954-05-19', 12366598741233),
(02417866145, 'Lima', 'Elias Benício Eduardo', '498805645', '1946-07-05', 12355578941569
);
```

Insert declarativo

O INSERT é classificado como declarativo quando especifica as colunas que receberão os dados da lista de valores Veja o próximo exemplo:

```
-- Insert declarativo
INSERT INTO clientes (cpf, sobrenome, nome, rg, datanasc, tituloeleitor) VALUES
(50956475329, 'Melo', 'Breno Oliver', '351224683', '1995-10-13', 12345678912345),
(33071211627, 'Paz', 'Fábio Luan da', '457732507', '1982-04-24', 12365498745633),
(11882181077, 'Silveira', 'Tereza Manuela Rita', '122931737', '1943-07-20', 12332145698777),
(13335743393, 'Castro', 'Gabrielly Ester Sabrina', '250821722', '1950-11-26', 12344578965254),
(86532048224, 'Santos', 'Elias Julio', '219901053', '1998-10-25', 1239986657411),
(95640167114, 'Fernandes', 'Helena Luana', '129121988', '1976-03-22', 12312312348874),
(80050797450, 'Neves', 'Bruna Clara', '331973091', '1958-05-04', 12399845676355),
(43343748161, 'Silva', 'Alessandra Stefany', '218698252', '1987-07-21', 12377745896313),
(66725468713, 'Gomes', 'Pedro Henrique', '146895654', '1954-05-19', 12366598741233),
(02417866145, 'Lima', 'Elias Benício Eduardo', '498805645', '1946-07-05', 12355578941569
);
```

Dicas

- Para acrescentar novas linhas de dados em uma tabela, utilizamos o comando **INSERT**;

- No **INSERT posicional** não é necessário mencionar o nome dos campos, porém será obrigatório incluir todos os campos, exceto o campo auto numerável Já no INSERT declarativo, os campos deverão ser informados no comando;

- Quando utilizamos a instrução INSERT dentro de aplicativos, **stored procedures** ou **triggers** deve ser usado o INSERT declarativo, pois, se houver alteração na estrutura da tabela (inclusão de novos campos), ele continuará funcionando, enquanto que o INSERT posicional provocará erro.22

Comando Select

O comando **select** é bastante simples e, serve para você, mostrar os dados da tabela Sua sintaxe pode ser utilizado de algumas formas, como:

SELECT * FROM NOME_DA_TABELA;

Exemplo: **SELECT * FROM clientes;**

Nota 1: o asterisco indica que você deseja mostrar todos os campos, ou seja, cpf, sobrenome, nome, rg, datanasc e tituloeleitor.

Exemplo: SELECT cpf, nome, datanasc FROM clientes;

Nota 2: neste caso, você está indicando que deseja mostrar somente cpf, nome e datanasc. Isso é interessante, pois a consulta é executa de forma mais rápida, já que você está trazendo menos informações da tabela.

Comando Update

Para alterarmos uma informação contida numa tabela do banco de dados, utilizamos o comando update. Ele atualiza dados de um registro ou de um conjunto de registro.

```
1 UPDATE clientes
2     SET nome = 'Breno Olliver'
3     WHERE cpf = 50956475329;
```

Sintaxe:

UPDATE nome_da_tabela
    SET <nome_da_coluna = novo_conteúdo>
    WHERE condicao

nome-tabela representa o nome da tabela cujo conteúdo será alterado.
nome-coluna representa o nome da(s) coluna(s) terão seus conteúdos alterados com o novo valor especificado.
condição representa a condição para a seleção dos registros que serão atualizados. Esta seleção poderá resultar em um ou vários registros. Neste caso a alteração irá ocorrer em todos os registros selecionados.

Comando Delete

O comando delete é utilizado para excluir linhas de uma tabela.

Sintaxe:
DELETE FROM <nome_da_tabela> **WHERE** <condição>

```
-- exclui o registro cujo cpf seja igual a 02417866145
DELETE FROM clientes
    WHERE cpf = 02417866145;

-- ou
-- exclui todos os dados da tabela clientes;
DELETE FROM clientes;
```

Quando vamos deletar qualquer registro, devemos nos lembrar da **Integridade Referencial**. Este conceito determina que um registro não pode fazer referência a um outro registro do banco de dados que não existe.

Interface phpMyAdmin

localhost / 127.0.0.1 | phpMyAdmi

localhost/phpmyadmin/

phpMyAdmin

Recente Favoritos

- Novo
- escola
- information_schema
- mysql
- performance_schema
- phpmyadmin
- sakila
- sys
- vendas
- world

Servidor: 127.0.0.1

Bancos de dados | SQL | Status | Contas de usuário | Exportar | Importar | Configurações | Log binário | Mais

Configurações gerais

Collation de conexão do servidor MySQL: utf8mb4_unicode_ci

Mais configurações

Configurações do Tema

Idioma *(Language)* Português (Brasil) - Portuguese (Brazil)

Tema pmahomme Ver todos

Servidor de banco de dados

- Servidor: 127.0.0.1 via TCP/IP
- Tipo de servidor: MySQL
- Conexão do servidor: O SSL não está sendo usado
- Versão do servidor: 8.0.45 - MySQL Community Server - GPL
- Versão de protocolo: 10
- Usuário: root@localhost
- Charset do servidor: UTF-8 Unicode (utf8mb4)

Servidor web

- Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12
- Versão do cliente de banco de dados: libmysql - mysqlnd 8.2.12
- Extensão do PHP: mysqli curl mbstring
- Versão do PHP: 8.2.12

phpMyAdmin

- Informações da versão: 5.2.3 (atualizado(a))
- Documentação

Console